



RAPPORT DE PSC - MEC N°17

GÉNÉRATION DE MÉTÉO LOCALE ET DYNAMIQUE POUR LE JEU VIDÉO

2022-2023

Marc BOËLLE, Mathieu POIRÉE, Charles RÉMY,
Emile ZACCOMER



INSTITUT
POLYTECHNIQUE
DE PARIS



REMERCIEMENTS

Nous tenons avant tout à exprimer notre reconnaissance à plusieurs personnes dont la contribution a été particulièrement importante pour notre projet.

Nous remercions ainsi Catherine Rolland, chef de projet du GameLab de l'École polytechnique, qui nous a accompagnés tout au long de notre PSC pour nous conseiller, nous orienter et nous mettre en contact avec le GameLab et le studio Asobo. Son soutien nous a été d'une grande aide pour le développement de notre projet.

Nous remercions également Stéphane Bordas pour nous avoir proposé ce sujet d'étude et avoir accepté de se rendre disponible pour des rencontres afin de préciser certains points de notre PSC. Ces entrevues nous ont permis de recadrer notre sujet et de nous mettre d'accord sur la direction à prendre.

Nous voulons également remercier Jean-Marc Chomaz, du LadHyX, qui nous a encadrés tout au long de notre projet et a su nous aiguiller pour recentrer, préciser et creuser nos pistes de recherche lorsque nous nous retrouvions face à un grand nombre d'approches possibles. Nous le remercions pour ses remarques constructives qui nous ont permis d'avancer.

Enfin, nous remercions les développeurs du GameLab que nous avons pu rencontrer lorsque nous faisons face à des obstacles pendant l'implémentation sur Unity de notre moteur. Leur aide a été précieuse.

RÉSUMÉ

Notre Projet Scientifique Collectif (PSC) a pour objectif la génération informatique de météo locale et dynamique. Il répond à une demande de la Direction Générale de l'Armement (DGA) qui cherche à améliorer son simulateur d'entraînement en y implémentant un système de météo local. Le caractère local de la météo a pour objectif de permettre aux militaires la prise en compte de la météo dans leurs décisions et donc de se rapprocher au plus près possible des conditions de combat réelles.

Conséquences du mouvement de l'air et de l'eau dans l'atmosphère et dépendant de nombreux paramètres, les phénomènes météorologiques sont souvent imprévisibles et chaotiques. Ces difficultés font donc de leur simulation un enjeu intéressant. Nous avons étudié différentes manières de générer une météo locale et dynamique, afin de trouver une approche qui puisse satisfaire au mieux les contraintes de notre cahier des charges (rendu à grande échelle, aspect réaliste, calculs réalisables sur les ordinateurs de la DGA).

Après avoir exploré puis écarté les modèles simulant les lois physiques régissant la météo à cause d'une complexité en calculs trop grande pour une simulation en temps réel, nous proposons un système de génération de météo locale et dynamique via des méthodes de génération procédurale. Notre méthode repose sur l'utilisation d'un bruit de valeur (value noise) transformé et animé par des opérations mathématiques pour reproduire l'apparence des nuages. La gestion de l'éclairage et des ombres est rendue possible grâce à l'outil de rendu Volumetric Clouds. Afin de permettre le test de l'expérience utilisateur, nous avons implémenté cette solution dans un démonstrateur sous Unity.

TABLE DES MATIÈRES

1	Introduction	6
1.1	Présentation du sujet et contexte de l'étude	6
1.2	Paradigme de départ	8
2	Approche physique	9
2.1	Méthode	9
2.2	Résultats	10
2.3	Bilan	12
3	Approche par automates cellulaires	14
3.1	Méthode	14
3.2	Résultats	14
3.3	Bilan	15
4	Génération procédurale: bruit de valeur et utilisation de shaders sur Unity	17
4.1	Principe des shaders	17
4.2	Résultats	19
4.3	Bilan	21
5	Utilisation du pipeline de rendu HDRP	23
5.1	Fonctionnement du pipeline de rendu HDRP	23
5.2	Résultats	24
5.3	Bilan	26
6	Implémentation de la pluie	28
6.1	Première étape : Pluie globale et statique	28



6.2	Deuxième étape: Pluie locale et dyanmique	30
6.3	Résultats	30
7	Conclusion	32
7.1	Résultat final	32
7.2	Réalisation des objectifs	33
7.3	Pistes d'amélioration	33

1

INTRODUCTION

1.1 PRÉSENTATION DU SUJET ET CONTEXTE DE L'ÉTUDE

Notre Projet Scientifique Collectif (PSC) a pour objet la réalisation d'un moteur de météo locale et dynamique en temps réel dans un simulateur ainsi que l'implémentation des effets des conditions météorologiques sur l'environnement et les joueurs.

Ce projet est en partenariat avec la Direction Générale de l'Armement (DGA) qui développe des simulateurs tactiques dans le but d'entraîner les troupes françaises. La DGA nous propose d'aborder une problématique spécifique à l'un de ces simulateurs, dont les spécificités sont les suivantes: les participants sont placés sur un champ de bataille virtuel et contrôlent chacun un personnage à la première personne. La partie se joue sur une carte de 500km par 500km, en réseau local. Le simulateur supporte la gestion de 1000 joueurs et est susceptible de tourner sur une durée d'environ 10 heures.

La reproduction la plus fidèle possible de la réalité est un enjeu majeur pour optimiser l'immersion et l'utilité de ces simulations, car les participants seront poussés à intégrer dans leur prise de décision des aléas qu'ils sont susceptibles de rencontrer en situation réelle. En opération, de nombreux scénarios font intervenir une situation météorologique : anticipation de la durée des manœuvres, prise en compte de la perte de visibilité en cas de mauvais temps, ralentissement en cas de pluie... L'ajout de la météo permet de diversifier les situations qu'un directeur de simulation peut faire rencontrer aux militaires.

Le simulateur actuel de la DGA ne dispose pour l'heure pas de moteur de météo locale et dynamique. Le directeur de simulation doit manuellement changer les conditions météorologiques, et ne peut appliquer ces changements qu'à l'ensemble de la carte de 50 000 km², ce qui rend impossible la prévision de l'évolution de la météo par le joueur. De plus, cela empêche la mise en place d'asymétries météorologiques entre les participants.

Face à cet enjeu, notre objectif final est de réaliser un démonstrateur de météo locale en temps réel dans lequel il est possible pour les participants d'observer l'évolution de la météo et de ses effets sur l'environnement. Le projet est effectué en contact régulier avec Stéphane Bordas, notre tuteur de la DGA. Nous lui présentons régulièrement l'avancée du développement de notre programme et discutons de nos choix et partis pris pour que l'objectif final soit à la fois réalisable par notre groupe et proche des attentes de la DGA. Ce démonstrateur est indépendant du simulateur de la DGA, et la problématique de la compatibilité, dans la perspective d'intégrer notre moteur au simulateur de la DGA se fera après le développement de cet outil.

Les principales contraintes de notre moteur météo sont les suivantes:

- permettre au joueur de comprendre facilement quelle est la météo actuelle afin de pouvoir réagir en conséquence
- pouvoir observer la carte de haut (“God view”) afin que le directeur de simulation puisse observer l’évolution de la météo globalement
- pouvoir utiliser un personnage jouable pour tester les effets de la météo sur l’environnement et le joueur
- permettre au directeur de simulation de choisir les conditions météorologiques pour s’entraîner dans des situations particulières
- avoir une complexité acceptable pour que le démonstrateur puisse être utilisé sur un ordinateur classique

Elles sont résumées dans le diagramme ci-dessous:

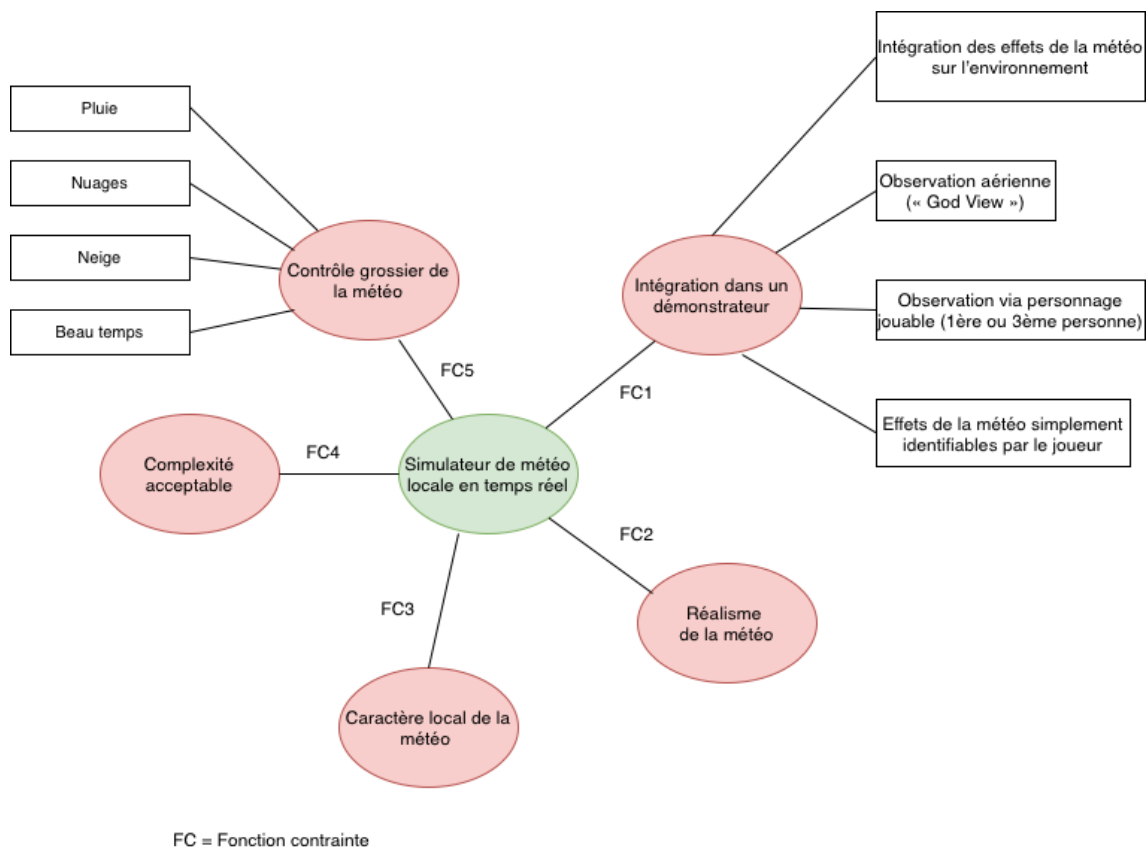


Figure 1: Principaux objectifs et contraintes de notre moteur météo

1.2 PARADIGME DE DÉPART

Étant donné que nous n'avons pas d'approche imposée par la DGA, nous avons eu la liberté de choisir notre propre approche. Pour cela, nous avons commencé par dresser une liste des différents paramètres météorologiques qui ont un impact significatif sur la simulation et qui sont pertinents pour le joueur. Nous avons retenu la **visibilité**, qui peut être altérée par les nuages, les **effets de lumière**, la **pluie**, le brouillard, la neige, ainsi que la vitesse de déplacement qui peut être affectée par la pluie qui rend le sol glissant.

Nous avons fait le choix de baser notre système de météo sur une **carte de couverture nuageuse**. Tout d'abord, les nuages sont des éléments visuels pour le joueur qui lui permettent d'identifier l'état de la météo. De plus, les nuages sont liés à de nombreux phénomènes atmosphériques qu'ils rendent possibles. Ainsi ce choix nous permet de fonder le reste des paramètres qui nous intéressent à partir de la carte de couverture nuageuse. Par exemple, plus la couverture nuageuse est importante, et plus la luminosité sera faible. La pluie apparaîtra aux endroits où la couverture nuageuse est importante. L'utilisation de la carte de couverture nuageuse nous permettra d'une part d'assurer la localité de la météo et d'autre part de déterminer les zones où les autres phénomènes météorologiques devront être générés.

Nous avons ensuite identifié les principaux enjeux de la création de la carte de couverture nuageuse. Les deux premiers que nous avons identifiés sont, d'un côté la gestion du **déplacement des nuages** et de l'autre la reproduction des **formes complexes des nuages** causées par une succession de perturbations partiellement chaotiques régies par les lois de la thermodynamique.

2

APPROCHE PHYSIQUE

L'approche, pour générer la carte de couverture nuageuse, par laquelle il nous semblait le plus naturel de commencer, est la simulation de lois physiques régissant les phénomènes météorologiques.

2.1 MÉTHODE

Nous nous sommes intéressés à des articles académiques abordant le sujet de la simulation de la météo à partir de conditions initiales données et des lois physiques de mécanique des fluides.

L'article "*A Method for Modeling Clouds based on Atmospheric Fluid Dynamics*" [1] présente une méthode de modélisation de nuages basée sur la **dynamique des fluides dans l'atmosphère**. Les auteurs discrétisent l'espace en mailles tridimensionnelles élémentaires et appliquent des équations d'évolution impliquant notamment la pression, l'advection par l'écoulement du fluide et la transition de phase vapeur-liquide. Ils effectuent leur simulation dans un espace de $256 \times 256 \times 40$ mailles, avec une largeur de maille de 20 m. Cette taille très réduite leur permet d'obtenir un temps de simulation acceptable (une trentaine de secondes). L'article "*A Simple, Efficient Method for Realistic Animation of Clouds*" [2] présente une manière simplifiée d'aborder la dynamique de formation des nuages avec des règles de transition simples entre trois états: nuage, transition de phase et vapeur. Dans les deux articles, les auteurs soulignent cependant que l'augmentation de l'espace et du nombre de mailles élémentaires entraîne une **augmentation** très importante du **temps de calcul**. Ainsi, cette méthode n'est pas adaptée à notre utilisation dans le cadre d'une simulation à grande échelle en temps réel.

La thèse d'Antoine Webanck [3] conforte notre volonté d'écarter un modèle utilisant la simulation de lois physiques discrétisées : "*Une approche permettant d'obtenir des formes nuageuses réalistes consiste à simuler informatiquement les lois thermodynamiques qui permettent l'évaporation de l'eau, la convection et les vents qui brassent les masses d'air humide ainsi que la condensation de l'eau lorsque les masses d'air sont saturées. Ce type de simulation fonctionne bien lorsque la quantité de masses d'air simulées reste faible, c'est-à-dire si le volume simulé et la finesse de sa discrétisation sont restreints. Dans le cas contraire, le coût de calcul passe mal à l'échelle et devient très rapidement prohibitif. Les simulations météorologiques mobilisent ainsi des supercalculateurs pendant des jours afin de prédire grossièrement le comportement des masses d'air, avec une fiabilité limitée spatialement et temporellement. Le niveau de détail et les dimensions des nuages sont donc limités par la puissance de calcul, et leur comportement chaotique dégénère rapidement avec les itérations de la simulation.*"

Pour pallier au problème de la complexité, nous décidons d'explorer une piste consistant à simplifier les équations d'évolution du modèle. Nous avons décidé de **modifier un modèle** développé dans la

thèse d'Antoine Webanck qui consiste à simuler le déplacement des nuages à partir de l'implémentation de l'équivalent d'une énergie potentielle également appelée "fonction de coût", dépendant notamment du relief du terrain et de la vitesse du nuage relativement au vent.

2.2 RÉSULTATS

Nous avons implémenté le modèle en programmation orientée objet: les nuages sont tous des **objets ponctuels d'une classe nuage**. Celle-ci contient notamment la position et la vitesse du nuage. Le terrain est découpé en mailles bidimensionnelles, chacune pouvant contenir un nuage. Nous avons modifié la méthode d'évolution décrite dans la thèse d'Antoine Webanck. Il propose de fixer la position initiale et finale de chaque nuage et de minimiser la fonction de coût sur l'ensemble du parcours. Puisque nous ne désirons pas spécifier la position finale du nuage, nous opérons plutôt le déplacement à chaque *frame* (image) en choisissant pour chaque nuage la position suivante qui minimise la fonction de coût.

A chaque pas de temps, le déplacement d'un nuage se fait en évaluant la fonction de coût associée aux déplacements vers chaque position suivante admissible pour le nuage. Les positions admissibles sont les cases adjacentes à la maille correspondant à la position actuelle du nuage. La position suivante retenue est celle qui **minimise la fonction de coût**.

Cette fonction de coût dépend de l'altitude du terrain autour du nuage et du vent. Elle a été construite de la manière suivante: on considère le mouvement possible de la case actuelle de position p_{actuel} à une case adjacente de position p_{test} . Soit v_{nuage} la vitesse instantanée correspondant au déplacement du nuage de p_{actuel} vers p_{test} . On a :

$$v_{nuage} = \frac{p_{test} - p_{actuel}}{\Delta t}$$

où Δt est le pas de temps entre deux images. Soit θ l'angle entre le vecteur nuage v_{nuage} et le vecteur vent v_{vent} . En considérant que l'altitude du terrain est stockée dans le tableau h , la formule du coût associé au déplacement considéré est donnée par la formule suivante:

$$C(p_{actuel}, p_{test}, v_{vent}, h) = \|v_{nuage} - v_{vent}\| \times (2 - \cos(\theta)) + k \times h(p_{test})$$

où k est une constante que nous déterminons empiriquement de manière à assurer l'équilibre entre les deux parties de la fonction de coût.

La prise en compte de l'altitude du terrain fait qu'il est plus coûteux pour un nuage de prendre de l'altitude pour passer au-dessus d'une montagne que de la contourner. La justification physique est la suivante: si un nuage est forcé de monter à cause du terrain, alors il se retrouve entouré de masses d'air de densité inférieure et une force orientée vers le bas s'applique sur lui. La pénalisation de l'altitude permet de rendre compte de ce phénomène. La première partie de la formule est la prise

en compte du vent. Avec sa contribution, les déplacements dans le sens du vent seront privilégiés alors que les déplacements à l'encontre du vent seront plus difficiles.

Nous avons implémenté ce modèle en Python. Ci dessous des captures d'écrans de différentes simulations effectuées en utilisant la fonction de coût.

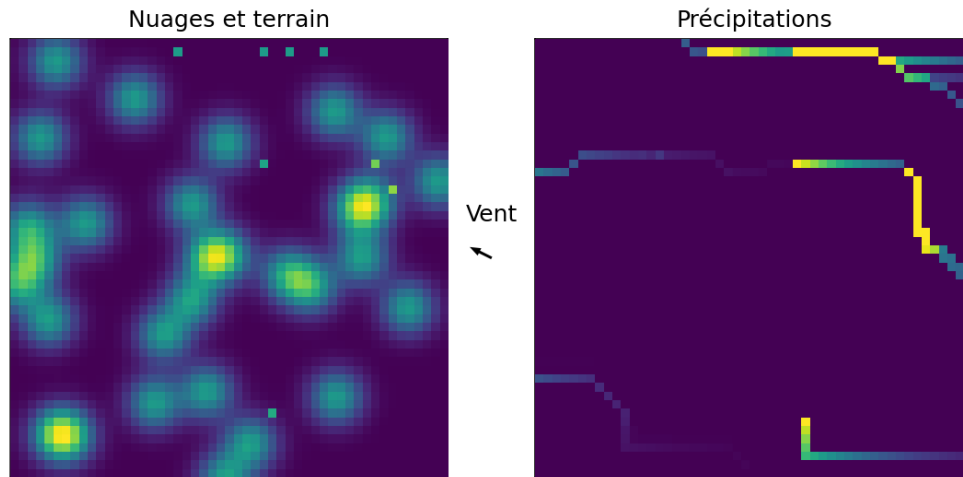


Figure 2: Déplacement de nuages ponctuels par minimisation d'une fonction de coût

Dans la figure 2, le terrain est représenté en couleur à gauche (jaune pour les hautes altitudes) ainsi que les nuages (pixels colorés isolés) et les précipitations sont représentées à droite (jaune pour les fortes précipitations). Les nuages se déplacent en temps réel sur le terrain généré aléatoirement de sorte à minimiser leur fonction de coût à chaque étape. La précipitation est ici simplement utilisée pour **représenter la trajectoire des nuages** : le séchage de la pluie tombée est choisi linéaire avec le temps et avec un coefficient arbitraire. On peut voir grâce à leur trajectoire que les nuages **évitent les zones de haute altitude** et vont globalement **dans la direction du vent**.

La figure 3 correspond à une version en 3 dimensions de la simulation précédente. Les nuages sont représentés par les points rouges se déplaçant dans l'espace.

Dans cette version 3D, nous avons ajouté un coût infini pour les cubes du maillages correspondant au montagne afin que les nuages ne puissent pas les traverser.

La figure 4 montre une généralisation de la méthode afin d'avoir des nuages étendus. Pour cela, nous reprenons l'implémentation précédente et les nuages sont maintenant des agrégats de points. On laisse ensuite chaque point du nuage se comporter selon les mêmes lois que dans la simulation précédente.

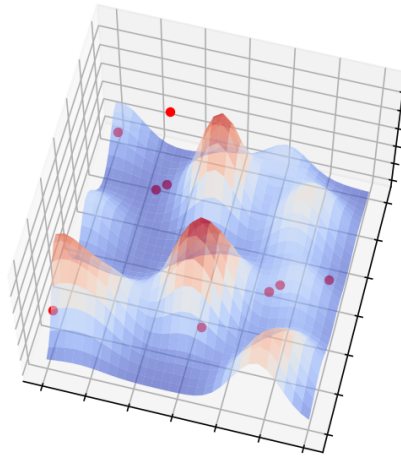


Figure 3: Déplacement de nuages ponctuels sur un terrain en trois dimensions

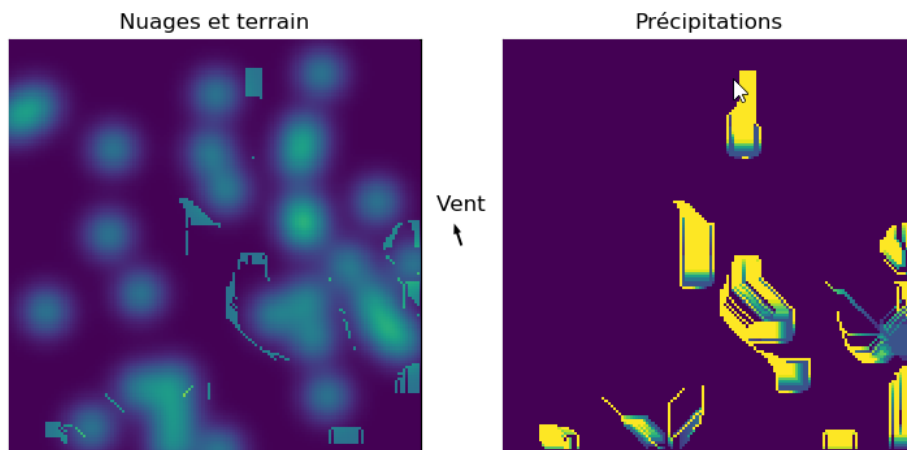


Figure 4: Déplacement de nuages étendus formés par agrégation de nuages ponctuels

2.3 BILAN

Nous avons obtenu une première version de déplacement des nuages sur la carte. Les nuages se comportent comme nous l'avions attendu: **ils suivent globalement la trajectoire du vent** mais sont susceptibles d'être **déviés légèrement par le relief** du fait de la compétition entre les deux termes de notre fonction de coût. Un avantage de cette méthode est la **modularité** et surtout le **contrôle** sur la fonction de coût: en la modifiant, nous pouvons décider librement de la dynamique d'évolution des nuages. Ainsi, il est possible de prendre en compte d'autres paramètres pour proposer une simulation plus réaliste (coût pour le déchargement en eau d'un nuage, coût relatif à la pression,

coût dépendant de la température ambiante etc...). Cependant, cette approche fait apparaître des **limitations liées à la complexité** lorsque le nombre de nuages et la taille de la carte deviennent importants (or, nous visons une simulation sur une carte de 500km par 500km). De plus, cette méthode ne se généralise pas bien pour des nuages étendus. Lorsque les nuages sont représentés par un agrégat de point, chaque point étant indépendant des autres, leur déplacement n'est pas satisfaisant car on perd rapidement la forme de nuage.

Devant les problèmes de la méthode de simulation physique révélés par les simulations précédentes, nous avons décidé d'explorer l'approche probabiliste. Cette approche, présentée dans la partie suivante, nous permet d'appréhender la reproduction des formes complexes des nuages causées par une succession de perturbations partiellement chaotiques régies par les lois de la thermodynamique. Contrairement à celui qui vient d'être présenté, ce modèle prend en compte les **interactions entre particules du nuage** sans nécessiter l'implémentation de lois physiques. Il permet donc de représenter des nuages étendus plus réalistes qu'avec le modèle précédent.

3

APPROCHE PAR AUTOMATES CELLULAIRES

D'après ce qui a été dit précédemment, afin d'améliorer le réalisme des nuages étendus, nous voulons introduire une corrélation spatiale. L'idée que nous avons suivie est la suivante: un nuage peut apparaître dans une case du maillage en fonction de l'état des cellules environnantes. Ce paradigme d'étude invite à aborder le modèle de système dynamique discret que sont les **automates cellulaires**.

3.1 MÉTHODE

Les automates cellulaires ont été largement utilisés pour simuler des phénomènes biologiques, physiques et sociaux, et sont également utilisés en informatique pour résoudre des problèmes de calcul intensif. Ils sont donc également applicables pour l'étude des systèmes météorologiques.[4]

Les automates cellulaires sont des modèles mathématiques qui simulent des systèmes dynamiques en utilisant des grilles discrètes de cellules. Chaque cellule peut prendre une valeur discrète à partir d'un ensemble fini de valeurs possibles, par exemple 0 ou 1. Le fonctionnement de base d'un automate cellulaire est régi par des règles de transition qui vont déterminer comment les valeurs de chaque cellule évoluent dans le temps. Ces règles sont définies en fonction de l'état actuel de la cellule et de ses voisins dans la grille. Nous nous sommes donc intéressés aux automates cellulaires bidimensionnels.

3.2 RÉSULTATS

Nous avons créé une simulation Python 2D où le nuage est constitué d'un agrégat de pixels. L'état possible pour chaque pixel est "nuage" ou "non nuage". La règle d'évolution est la suivante: la probabilité qu'un pixel soit dans l'état "nuage" à un instant donné est une variable de Bernoulli dont la probabilité dépend de l'état de ses voisins à l'instant précédent: si tous ses voisins sont dans l'état "nuage" à l'instant précédent, il sera dans l'état "nuage"; si la moitié sont dans l'état "nuage", il aura une probabilité 0.5 d'être dans l'état nuage. Le vent est pris en compte par le déplacement de l'ensemble des nuages dans sa direction. On fixe une certaine probabilité d'extinction à chaque instant pour chaque carré de nuage afin de garder un certain réalisme.

La figure 5 montre trois captures d'écran présentant l'évolution de la simulation via l'approche probabiliste. Les cases en jaunes correspondent aux particules du nuage. A l'instant initial, la simulation est lancée avec une seule case jaune. Le nuage évolue ensuite selon les règles d'évolution décrites précédemment. Nous observons un comportement dynamique de la simulation. La corrélation spatiale

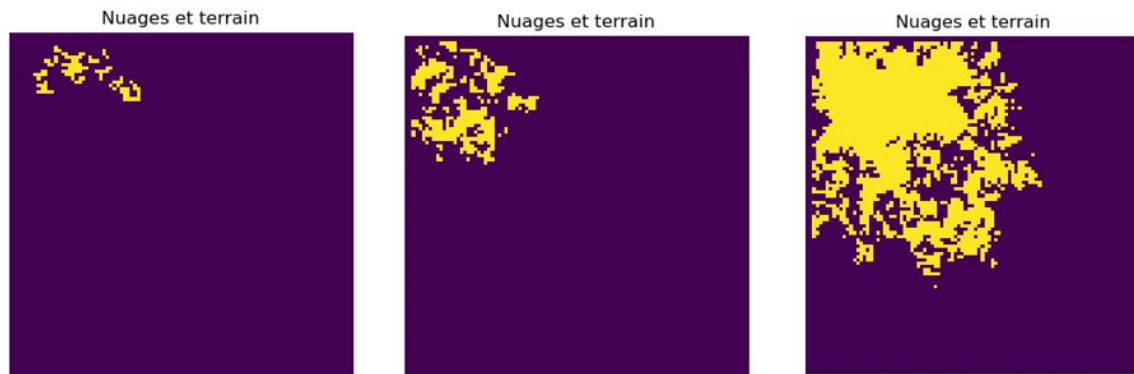


Figure 5: Captures d’écran de la simulation à différents instants

est bien respectée car le nuage se propage de proche en proche.

3.3 BILAN

Cette méthode permet de simuler une évolution des nuages due au hasard. L’approche probabiliste permet une première simulation de la déformation des nuages au cours du temps. De plus, les règles de transition entre les états “nuage” et “non nuage” permettent de former des agrégats de points ayant l’apparence de nuages. Cependant, elle est gourmande en temps dès que la carte dépasse quelques centaines de pixels. En effet, elle nécessite de calculer l’état de tous les pixels de la carte à chaque pas de temps.

Si nous n’étions pas freinés par le temps de calcul très long, nous pourrions complexifier le modèle en ajoutant des règles de transition plus sophistiquées et reproduisant mieux le comportement des nuages.

Dans l’optique de trouver une autre approche à considérer, nous avons rencontré des développeurs d’Asobo, entreprise qui a travaillé sur le jeu vidéo Microsoft Flight Simulator. Ce jeu possède actuellement le meilleur moteur de simulation de météo sur le marché. Lors de cette réunion, ils nous ont notamment expliqué leur méthode de génération des nuages. Ils génèrent d’abord une **carte de couverture nuageuse à grande échelle**, non détaillée, qui spécifie la quantité de nuages dans des zones de dizaines de km de côté. Ils gèrent ensuite l’aspect visuel local des nuages en combinant la carte de couverture nuageuse à grande échelle et une **texture de bruit à petite échelle**. Cette combinaison permet d’obtenir des nuages visuellement réalistes tout en contrôlant la météo avec la carte de couverture nuageuse.

Inspirés par cette idée, nous avons décidé de faire intervenir **plusieurs échelles** dans notre simulation. Nous avons conclu de l’approche physique et probabiliste que la gestion microscopique et macroscopique doivent être traitées séparément. En effet, si l’ensemble de la carte est traitée par des méthodes microscopiques, la complexité de calcul explose. De plus, la cohérence à l’échelle



macroscopique est difficilement contrôlable. Après avoir appréhendé le problème avec des simulations Python, il est cohérent de choisir un outil de rendu graphique plus puissant qui nous permettra à terme de créer un démonstrateur. Nous décidons donc de choisir Unity pour continuer nos simulations.

4

GÉNÉRATION PROCÉDURALE: BRUIT DE VALEUR ET UTILISATION DE SHADERS SUR UNITY

Nous avons besoin d'une très grande étendue spatiale pour nos nuages, et ils doivent pouvoir être modifiés au cours du temps. C'est pourquoi nous choisissons d'utiliser la **génération procédurale**. La génération procédurale (ou modèle procédural) est la création de contenu numérique à une grande échelle de manière automatisée répondant à un ensemble de règles définies par des algorithmes. Elle peut donc être utilisée pour la génération de météo [5].

Unity est un environnement de développement de jeux vidéo, basé sur un moteur de jeu en temps réel, qui utilise le langage C# pour créer des applications interactives en 2D et 3D, en utilisant des techniques de rendu avancées telles que l'éclairage dynamique, les shaders et la physique simulée.

4.1 PRINCIPE DES SHADERS

Les **shaders** [6] sont de petits scripts qui contiennent les calculs mathématiques et les algorithmes permettant de calculer la couleur de chaque pixel rendu.

Les calculs des shaders sont exécutés sur le GPU (Graphics Processing Unit) de l'ordinateur, ce qui permet à l'affichage de gagner en rapidité. En effet, le GPU est capable de générer le rendu d'images plus rapidement qu'un CPU (Central Processing Unit, processeur présent dans tous les ordinateurs) grâce à son architecture de traitement parallèle, qui lui permet d'effectuer de nombreux calculs simultanément.

Un **Shader Graph** [7] est un outil visuel qui permet aux utilisateurs de créer et de modifier des shaders personnalisés sans avoir à écrire de code. Il utilise un système de nœuds représentant des opérations mathématiques. Il est possible d'imbriquer les nœuds entre eux pour obtenir la texture voulue ainsi que pour créer des graphiques avancés et des effets visuels dans les jeux en temps réel.

Motivés par la rapidité de l'affichage des shaders et par les nombreuses possibilités qu'ils offrent, nous utilisons les shaders pour générer une forme de nuage et appliquons ces formes à un plan 2D dans le ciel. Avec l'utilisation des shaders, les enjeux sont multiples: obtention d'une couverture nuageuse locale, c'est-à-dire non identique d'une région à une autre, contrôle de la couverture nuageuse et déplacement des nuages sous l'effet du vent.

Nous utilisons donc la génération procédurale de bruit pour générer la forme de nos nuages. Or pour pouvoir reproduire la forme des nuages à l'aide d'un algorithme, nous devons connaître leurs caractéristiques. Une des caractéristiques importantes de la forme des nuages est qu'ils forment des

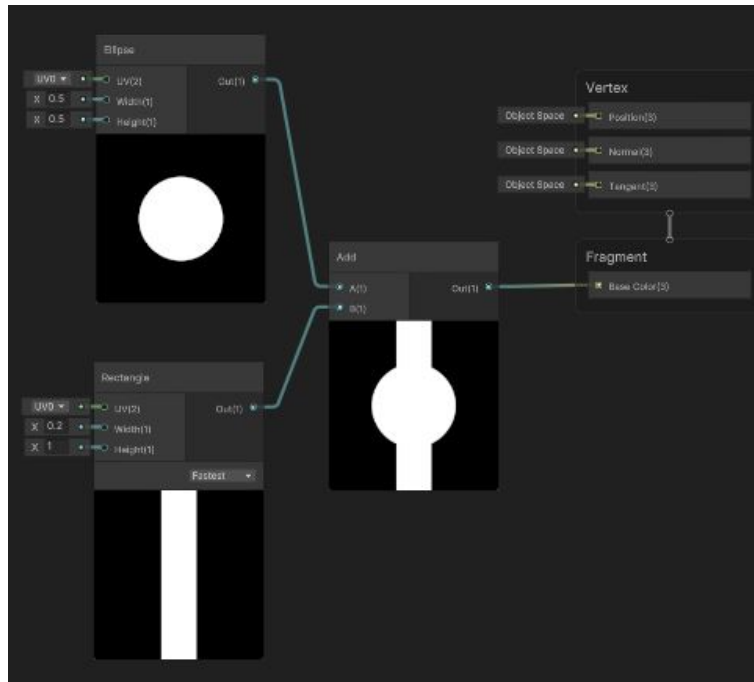


Figure 6: visualisation des noeuds dans Shader Graph : un noeud Ellipse, Rectangle et addition sont utilisés pour générer un rendu graphique simple

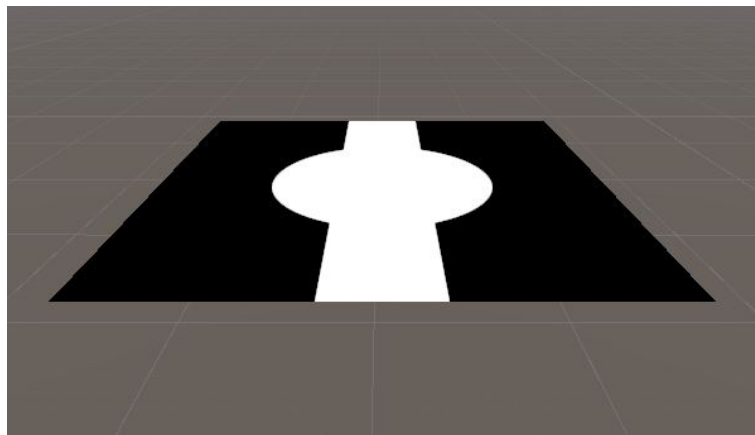


Figure 7: application de ce rendu sur un plan à deux dimensions

bancs, des **nappes** ou apparaissent sous la forme de **couches**. Dans tous les cas, il existe une **corrélation spatiale** dans leur forme. On ne peut donc pas utiliser la méthode la plus simple de génération de bruit consistant en une variable aléatoire suivant la loi de probabilité uniforme sur $[0, 1]$, car ce bruit ne présente pas de corrélation spatiale des variables aléatoires. Nous nous sommes donc penchés sur d'autres modèles de génération procédurale, et notamment le **bruit de valeur** ou *value noise*.

4.2 RÉSULTATS

A l'origine de notre texture nuageuse, nous avons utilisé un bruit de valeur [8] (simple/value noise) à deux dimensions. Le principe est le suivant: on utilise un ensemble de valeurs discrètes aléatoires uniformes (entre 0 et 1) réparties dans un tableau à deux dimensions, appelé seed (graine). On effectue ensuite une interpolation bilinéaire entre ces valeurs discrètes, permettant d'obtenir pour **chaque point** de notre espace à deux dimensions la valeur correspondante. Chaque valeur est ensuite convertie en niveau de gris (0 correspond au noir et 1 au blanc): on obtient une carte finale pouvant être utilisée comme base de nuages. Shader Graph donne la possibilité de gérer la fréquence spatiale du bruit de valeur. En combinant plusieurs octaves de bruit (des bruits dont la fréquence spatiale est multiple d'une fréquence de référence), nous avons introduit des détails à haute comme basse fréquence spatiale.

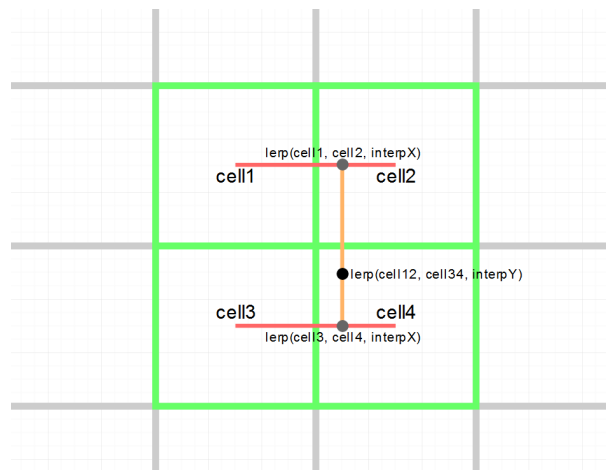


Figure 8: Principe du bruit de valeur

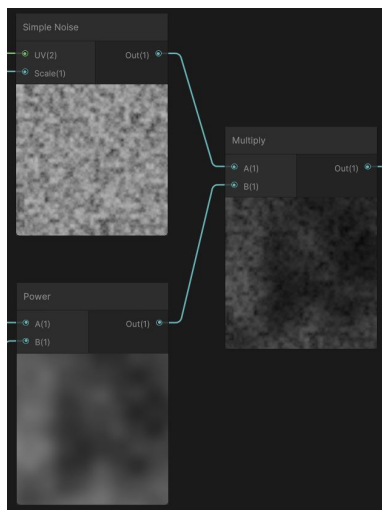
Dans la figure 8, on effectue une interpolation bilinéaire entre les valeurs discrètes situées au centre de *cell1*, *cell2*, *cell3* et *cell4*: une interpolation linéaire est d'abord effectuée entre les valeurs de *cell1* et *cell2*, puis entre *cell3* et *cell4*, et enfin entre le résultat de ces deux interpolations. Cela permet d'obtenir la couleur de tout pixel entre les 4 cellules, tel représenté dans la figure 9.

Nous avons d'abord mis en mouvement le simple noise pour introduire le vent. Pour reproduire la légère **érosion** et modification de la forme des nuages au cours du temps, nous avons multiplié entre eux deux nœuds de simple noise de même fréquence spatiale dont les vecteurs vitesse sont différents. Cela engendre une **légère modification de la texture au cours du temps**.

Pour introduire la localité de nos nuages, nous avons multiplié entre elles plusieurs octaves de bruit. Les **octaves basse fréquence** permettent d'introduire des différences de couverture nuageuse **sur des zones entières**. Nous obtenons une couverture nuageuse plus disparate, avec des zones couvertes et d'autres moins couvertes telles que dans la figure 10.



Figure 9: value noise (ou simple noise) dans Shader Graph sur Unity



(a) Interface Shader Graph



(b) Affichage dans la scène

Figure 10: Multiplication de deux octaves entre elles(une d'échelle 15 et une d'échelle 510)

Enfin, pour pouvoir contrôler la couverture nuageuse, nous avons branché le nœud de bruit à un nœud "power", qui met la texture à une puissance pouvant être contrôlée à l'extérieur du shader. Les valeurs du bruit étant comprises entre 0 et 1, l'augmentation de la puissance entraîne une diminution de ces valeurs et donc de la couverture nuageuse résultante, tel qu'affiché dans la figure 11.

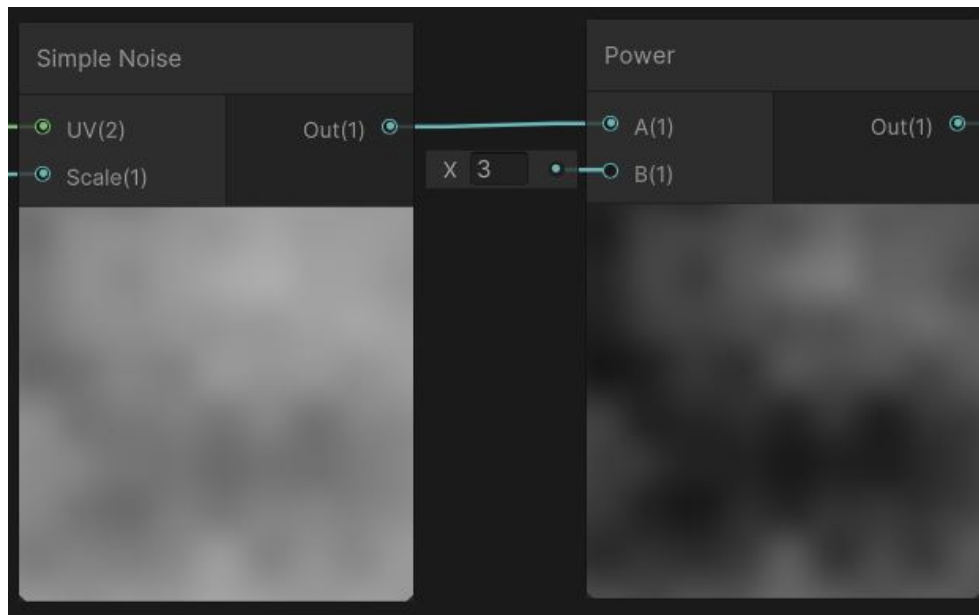


Figure 11: Couverture nuageuse mise à la puissance 3: on obtient une couverture nuageuse de même forme mais moins intense



Figure 12: Juxtaposition de deux zones de couverture nuageuse différente (puissance 1.5 à gauche, 0.9 à droite)

4.3 BILAN

Cette méthode nous a permis d'obtenir un premier rendu graphique de nuage. Nous avons le **contrôle** de la couverture nuageuse, une **évolution temporelle** des nuages ainsi qu'un caractère

local. En ce sens, l'utilisation des shaders a permis une grande avancée pour l'affichage des nuages.

Il existe néanmoins plusieurs points négatifs: d'abord, les nuages que nous avons générés sont en **deux dimensions**. Pour les types de nuages à faible extension verticale, cela ne pose pas de problème majeur. Cependant, pour les nuages à grande extension verticale comme les cumulonimbus (nuages d'orage), cette méthode n'est pas satisfaisante. De plus, l'un des objectifs du projet est de créer une météo qui ait un impact sur le joueur. Or avec la technique utilisée, c'est-à-dire l'application du shader sur un plan 2D, Unity ne permet pas de jouer sur la luminosité et d'introduire des zones d'ombre sur le terrain en dessous des nuages. En effet, il faudrait pour cela que seules les parties blanches, ou "nuageuses" du plan soient considérées comme opaques, ce qui est impossible. Les nuages générés sont donc pour l'instant uniquement visuels.

Suite à la présentation de mi-projet faite en présence de M. Chomaz, nous avons réfléchi à une méthode pour améliorer notre modèle et introduire un impact sur le joueur. Nous nous sommes donc mis à la recherche d'un outil permettant à nos nuages d'avoir une influence sur l'**éclairage** de la scène.

5

UTILISATION DU PIPELINE DE RENDU HDRP

Nous nous sommes donc penchés sur les outils d’affichage présents dans Unity. Pour afficher le contenu d’une scène, Unity utilise des pipelines de rendu (render pipelines). Le pipeline de rendu effectue une série d’opérations qui prennent le contenu d’une scène (objets, source de lumière) et l’affiche à l’écran. Il existe différents pipelines de rendu sur Unity. Pour notre projet jusqu’à présent, nous avons utilisé le pipeline de rendu intégré (built-in render pipeline). Il est conçu pour le rendu de scènes de taille petite à moyenne et fonctionne en rendant la géométrie et les matériaux un par un, dans l’ordre où ils sont définis dans la scène.

Pour améliorer les effets de lumière de nos nuages, nous devons faire en sorte qu’en présence d’une source de lumière, les nuages créent des ombres et empêchent donc la lumière de passer. Pour cela, nous décidons d’utiliser un autre pipeline de rendu: HDRP (High Definition Rendering Pipeline).

5.1 FONCTIONNEMENT DU PIPELINE DE RENDU HDRP

Outre la possibilité d’utiliser des unités physiques pour l’affichage des scènes (le lux par exemple), HDRP utilise la technologie de Ray Tracing pour calculer les ombres en temps réel, ce qui permet de créer des ombres douces et précises avec une grande qualité.

Le **ray tracing** [9] (ou ”suivi de rayons” en français) est une technique de rendu utilisée dans les moteurs de jeu pour simuler la façon dont la lumière interagit avec les objets d’une scène 3D (cf référence). Le principe est de lancer les rayons à partir de la caméra du joueur et non à partir des sources de lumière, puisque seuls ces rayons sont potentiellement utiles pour l’affichage de la scène. Lorsqu’un rayon lancé à partir de la caméra atteint une surface, il peut être réfléchi, réfracté, absorbé ou diffusé en fonction des propriétés de cette surface. Par le principe du retour inverse de la lumière, seuls les rayons qui atteignent à un moment une source de lumière sont ensuite générés pour afficher la scène.

Nous utilisons aussi le module **Volumetric Clouds** [10] intégré au pipeline de rendu HDRP. Ce module permet d’afficher des nuages en trois dimensions qui peuvent générer des ombres et obtenir des effets visuels physiquement réalistes. Il existe plusieurs modes de fonctionnement de ce module: un mode simple, générant automatiquement une carte de nuages dont différents paramètres peuvent être modifiés, comme la taille ou le type des nuages. Cependant, ce mode offre peu de liberté puisque la carte de nuages ainsi créée est uniforme et ne présente pas de caractère local. un mode avancé, qui permet de spécifier à l’aide de textures à deux dimensions en niveau de gris, la forme exacte des nuages que nous désirons. La texture est convertie par le module pour donner une forme à trois dimensions suivant le type de nuages. Le module Volumetric Clouds propose en effet trois types de

nuages: cumulus, alto stratus et cumulonimbus (l'extension verticale et la hauteur sont différentes pour chacun). Nous pouvons gérer indépendamment la forme et la texture de chaque type de nuages.

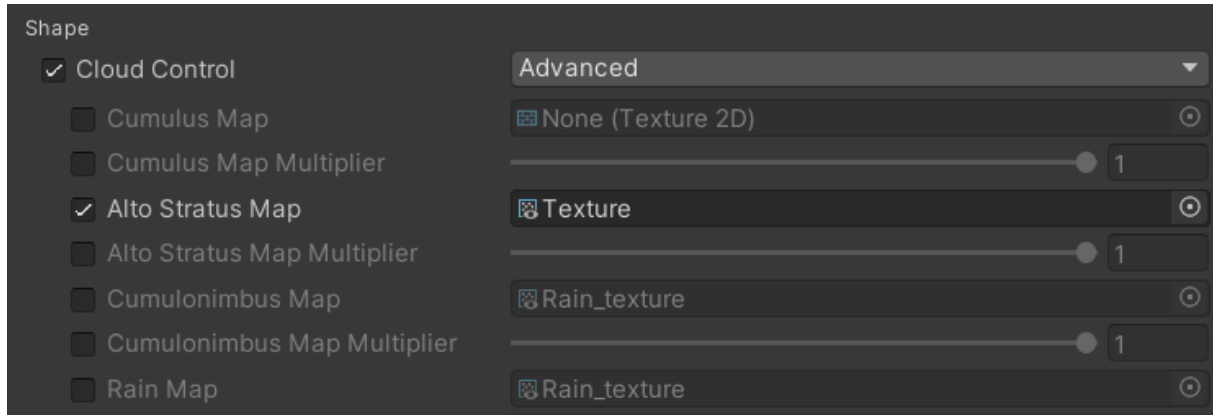
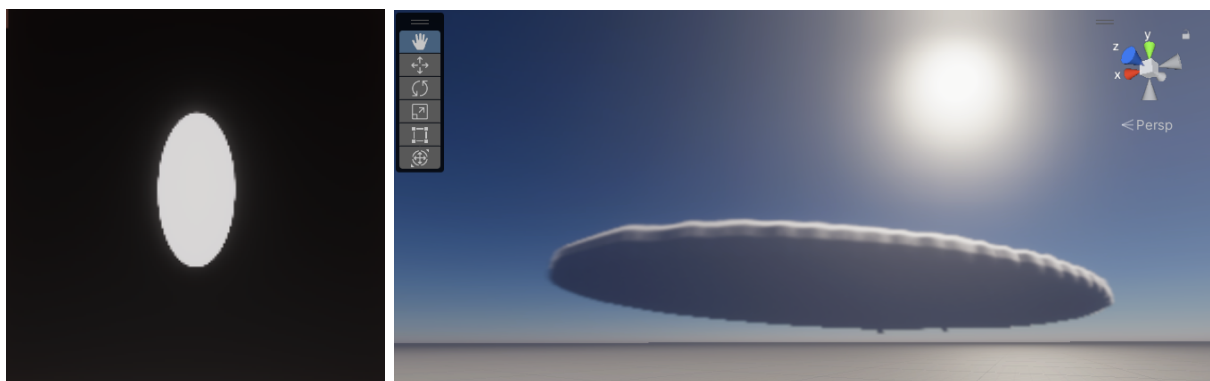


Figure 13: Interface du mode avancé du module Volumetric Clouds de Unity

Pour générer la texture 2D qui sera utilisée par le module Volumetric Clouds, nous décidons d'utiliser les formes de nuages que nous avons **précédemment créées à l'aide des shaders**. L'objectif est d'ainsi concilier les avantages de la méthode des shaders, à savoir **contrôle sur la météo** et **localité des nuages**, avec le rendu tridimensionnel et permettant des jeux de lumière réalistes du module Volumetric Clouds.

5.2 RÉSULTATS



(a) Image générée par le shader

(b) Rendu dans la scène par le module Volumetric Clouds

Figure 14: Comparaison du shader et de son rendu par le module Volumetric Clouds

Avec le concours des développeurs Unity du Game Lab de l'école, nous avons réussi à convertir l'image générée par les shaders de la partie précédente en une texture 2D lisible par le module Volu-

metric Clouds. La figure 14 fournit un exemple de la conversion d'une forme simple et reconnaissable. Le nuage obtenu à droite est tridimensionnel et son éclairage est modifié par le Soleil.

Les images ci-dessous montrent différentes simulations permettant de visualiser les résultats obtenus.

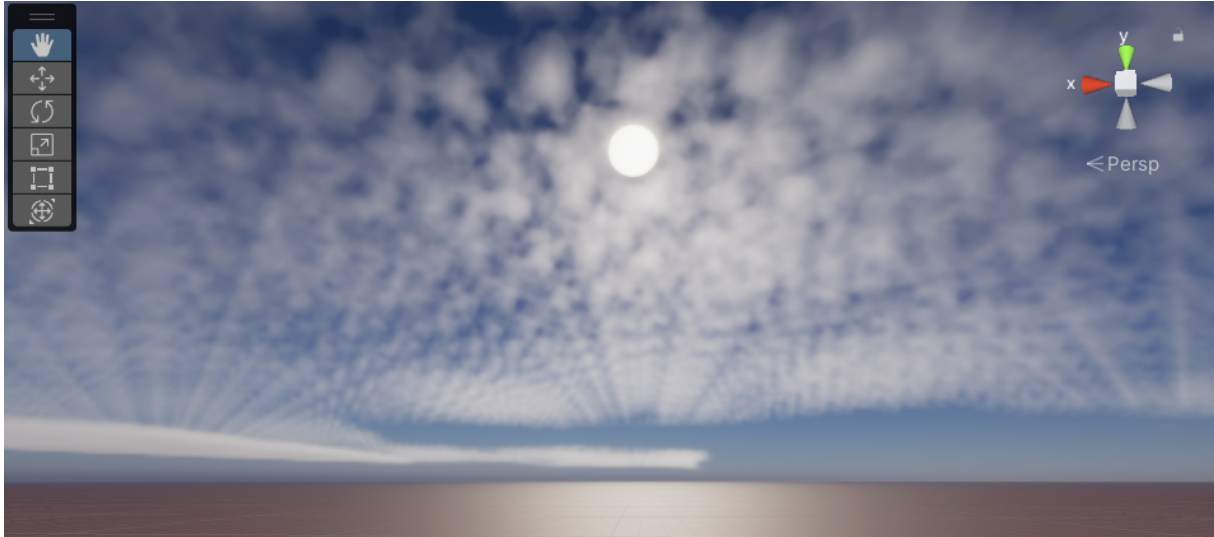


Figure 15: Création d'un nuage épars à partir de plusieurs octaves de bruit

La figure 15 montre le rendu de la superposition de deux bruits de fréquences différentes. Le bruit de basse fréquence donne la **forme globale** de la couverture nuageuse tandis que le bruit de fréquence élevée permet d'**introduire des détails** dans l'affichage du nuage. Il résulte un nuage clairsemé laissant le soleil apparaître entre les nuages. En jouant sur les fréquences et l'intensité des bruits utilisés, nous pouvons obtenir un grand nombre de formes de nuages et de types de temps.

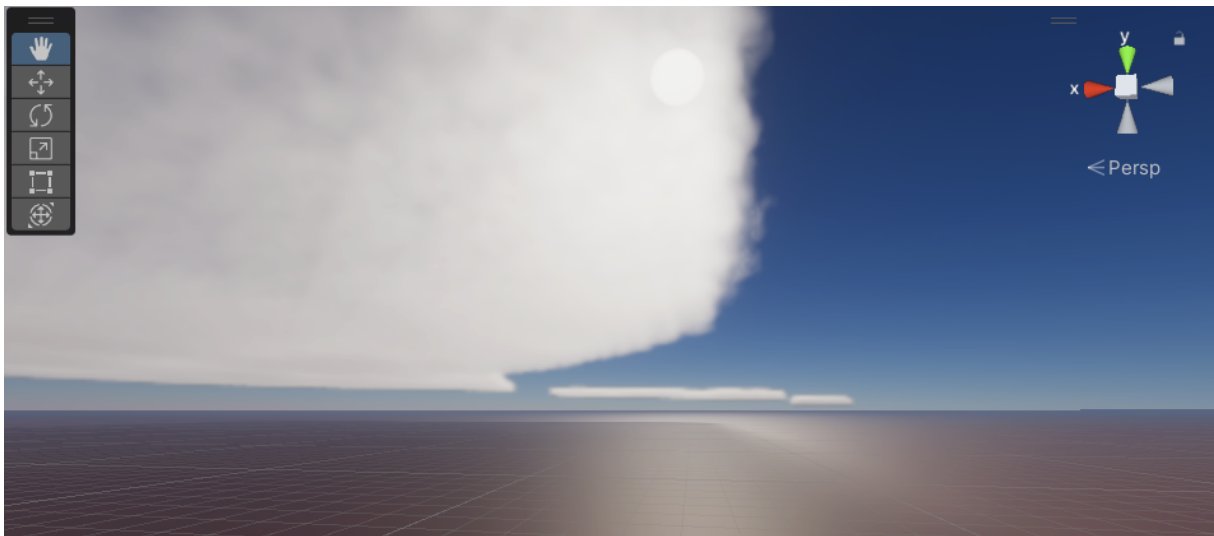


Figure 16: Visualisation de l'ombre des nuages sur le sol

La figure 16 met en évidence l'**ombre du nuage sur le sol**. Cette ombre est calculée par ray tracing (d'après le principe décrit en section 5.1) en fonction de la position du nuage par rapport au soleil. L'effet sur le joueur est une visibilité réduite sous un nuage opaque.



Figure 17: Visualisation du caractère local de la météo générée

La figure 17 met en évidence la **localité de la météo générée**. Si à l'endroit de la prise de vue la météo est clémente, l'utilisateur peut apercevoir à l'horizon des nuages orageux. Pour ce faire, nous utilisons deux shaders: un pour les nuages orageux (cumulonimbus) et un pour les nuages éparses (alto stratus), ce qui nous permet de contrôler précisément la forme de chaque type de nuage. Nous fournissons en entrée du module Volumetric Clouds les textures résultant de ces deux shaders. Nous obtenons donc finalement une météo différente en fonction de la position du joueur. Elle est repérable à grande distance, ce qui permet au joueur d'adapter son comportement.

5.3 BILAN

Avec l'utilisation du pipeline de rendu HDRP et du module Volumetric Clouds combinés aux shaders dont la méthode d'utilisation a été décrite en section 4, nous obtenons un affichage tridimensionnel et réaliste des nuages. L'affichage tridimensionnel est en effet géré par le module Volumetric Clouds en fonction du type de nuage choisi. L'affichage réaliste est rendu possible notamment grâce au ray tracing utilisé par HDRP qui permet de simuler le comportement physique de la lumière.

Le caractère local de notre méthode de génération de météo est encore amélioré avec la possibilité d'utilisation de **différents shaders** pour générer chaque type de nuage. Ainsi, nous respectons les formes et la taille habituelles de chaque type de nuage: imposantes pour les cumulonimbus, sous forme de couche striée pour les alto stratus et de forme boursouflée pour les cumulus.

Avec cette méthode, nous créons dans Unity un **démonstrateur** permettant à un joueur à **la première personne** de se déplacer et de visualiser le **caractère local de la météo**. Nous implémentons aussi l'option de "god view", permettant de voir l'état de la météo depuis le ciel. Cela permet au directeur de simulation d'avoir une vision d'ensemble de la météo.

Dans notre étude, nous nous sommes d'abord concentrés sur la génération des nuages. Maintenant que nous disposons d'une carte de couverture nuageuse, nous désirons **augmenter l'impact** de la météo sur le joueur. Nous décidons ainsi de prendre en compte l'un des effets de la météo visibles : la pluie. L'implémentation de cet effet sera décrit dans la section suivante.

6

IMPLÉMENTATION DE LA PLUIE

6.1 PREMIÈRE ÉTAPE : PLUIE GLOBALE ET STATIQUE

Pour implémenter de la pluie dans Unity, on utilise un système de particules qui est une technique graphique numérique basée sur le principe suivant : on place une entité émettrice et celle-ci génère des particules à divers instants, chacune d'entre-elles évoluant selon des paramètres prédéfinis, comme présenté sur la figure 18.

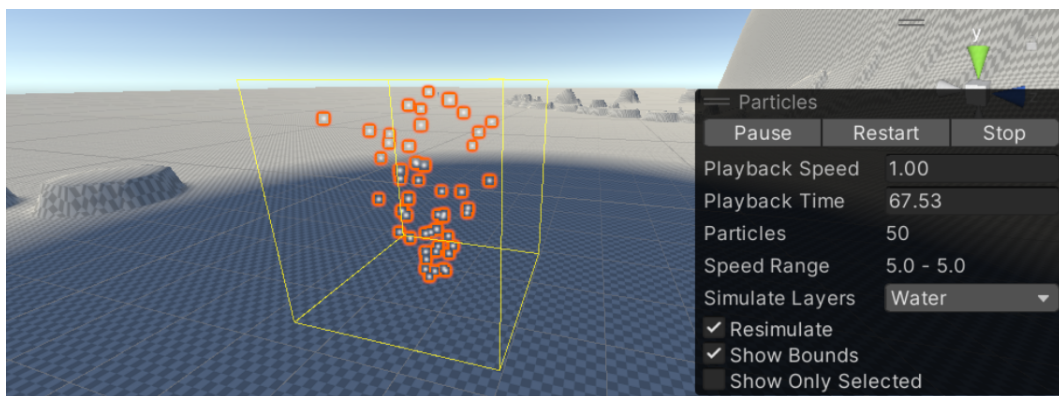


Figure 18: Système de particules basique sur Unity

Le réglage des paramètres des particules pour simuler de la pluie sur unity est très largement connu car il s'agit d'une des applications les plus classiques du système de particules. Nous nous sommes donc appuyés sur une paramétrisation (un "asset"), déjà accessible gratuitement, nommée "Rain Maker" [11] qui regroupe plusieurs fonctionnalités présentées sur la figure 19.

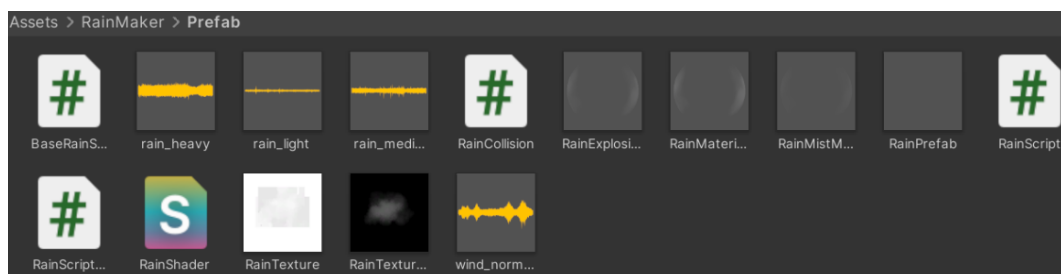


Figure 19: Contenu de l'asset Rain Maker : texture de gouttes d'eau, de brume et d'explosions de la goutte au sol, sons de pluie et de vent, shader de pluie et scripts

Classiquement, pour ne pas générer un nombre trop important de particules, on se restreint à générer des gouttes de pluie au-dessus du joueur et autour de lui dans un rayon de quelques mètres. Du point de vue du joueur en première personne, on a alors la même impression que s'il pleuvait partout. (cf. Figure 20)

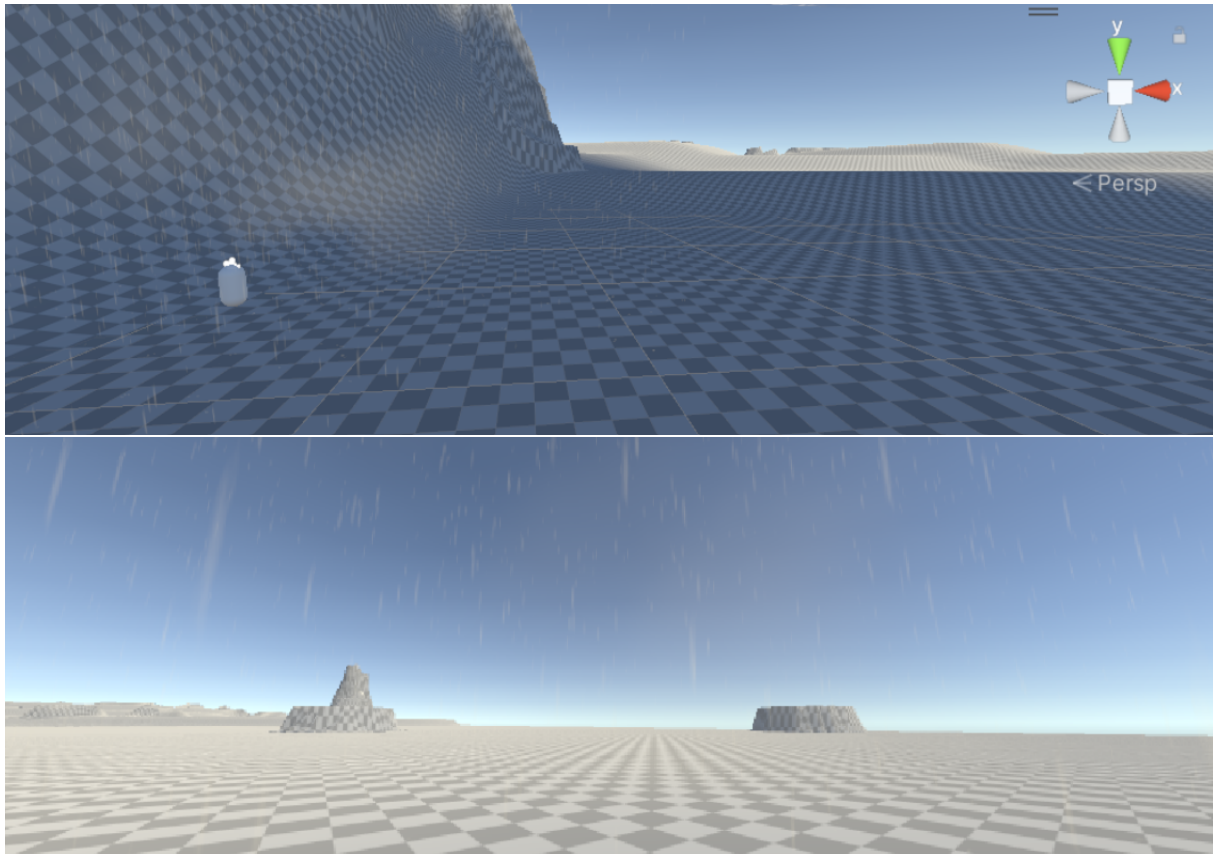


Figure 20: La pluie est générée au-dessus du joueur et non sur toute la carte, mais on ne le remarque pas à la première personne

Cette méthode est aussi compatible avec un mode multijoueur pourvu que chaque joueur possède son propre système de particules.

Cette méthode n'est pas optimale car elle ne permet pas de voir la pluie au loin. Cependant, elle permet de limiter le nombre de particules à afficher. D'autre part, dans la simulation, les variations d'intensité des précipitations ne sont pas abruptes au mètre près : puisque l'intensité des précipitations varient à l'échelle du kilomètre, le joueur peut quand même anticiper une averse.

Néanmoins, la pluie est, à ce stade, toujours globale et statique, c'est-à-dire que l'intensité des précipitations ne dépend ni de la position du joueur, ni du temps. Nous allons **modifier** l'asset existant afin de **rendre la pluie locale**.

6.2 DEUXIÈME ÉTAPE: PLUIE LOCALE ET DYNAMIQUE

L'objectif est alors d'ajouter et de modifier des scripts présents dans ce dossier pour rendre cette pluie locale et dynamique, en nous aidant d'une carte de pluie (*rain map*) spécifiant l'**intensité des précipitations** en fonction de la position avec des niveaux de gris (obtenue de manière analogue et cohérente avec la couverture nuageuse de la partie 4).

Le paramètre déterminant pour obtenir une pluie locale et dynamique est l'intensité des précipitations, qui est une variable dans les scripts de l'asset Rain Maker (plus précisément dans le script "BaseRainScript").

L'idée est donc de modifier la variable d'intensité de l'asset Rain Maker automatiquement à partir de la position du joueur et de la rain map. Il s'agit alors de rédiger un script qui mette à jour régulièrement la variable RainIntensity.

Le code que nous avons ajouté dans le script "BaseRainScript" réalise les tâches suivantes :

- récupération des coordonnées du joueur sur la carte relativement au terrain (prise en compte de la position et des dimensions du terrain)
- détermination du pixel correspondant à ces coordonnées dans la rain map (prise en compte des dimensions de la rain map)
- lecture du niveau de gris de ce pixel grâce à son code RGB
- mise à jour de l'intensité des précipitations au-dessus du joueur conformément au niveau de gris relevé tel que la valeur maximale (1) correspond à un pixel blanc et la valeur minimale (0) correspond à un pixel noir.

6.3 RÉSULTATS

Lorsque le joueur se déplace, l'intensité des précipitations varie selon les tendances décrites par la rain map (qui est dynamique grâce au shader graph). Pour illustrer ce résultat, nous choisissons une *rain map* simple composée de trois bandes de niveaux de gris croissant et nous l'affichons au-dessus du joueur en plus de la fournir en argument de notre script.

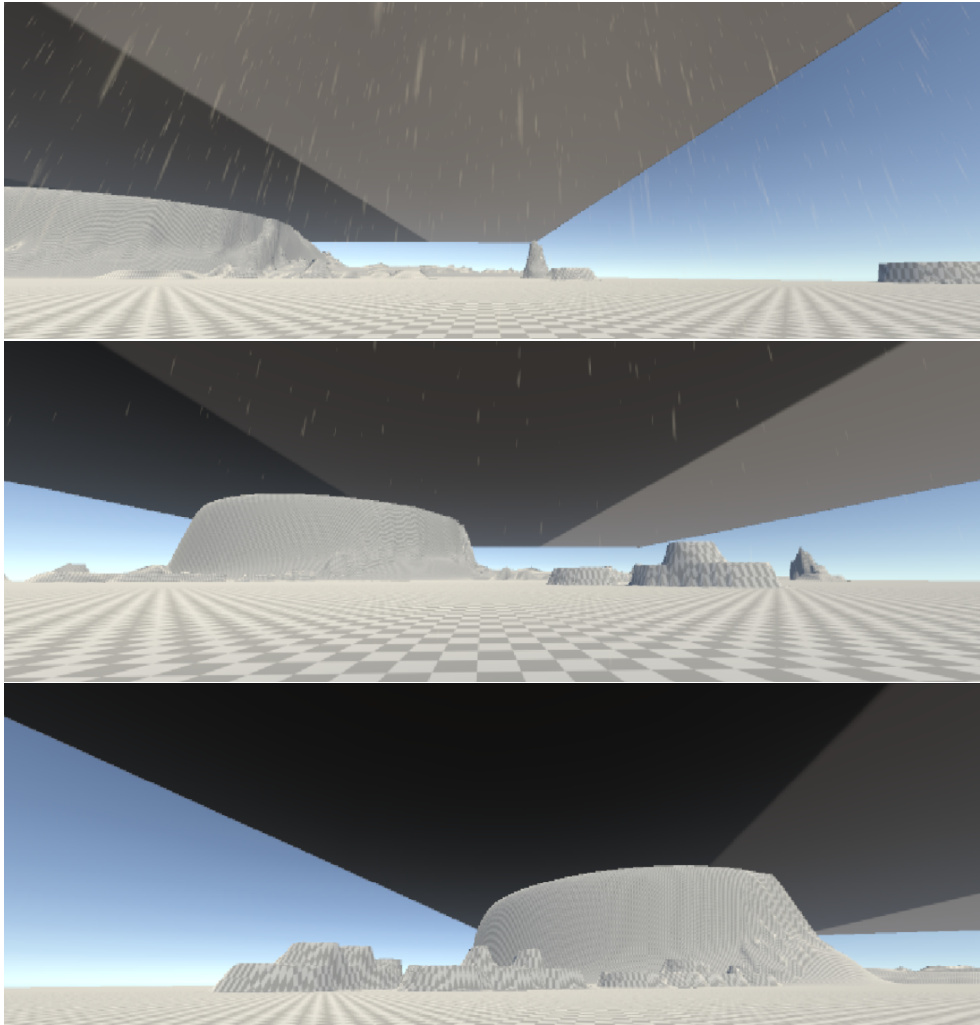


Figure 21: Variation de l'intensité de la pluie en fonction de la *rain map*; le noir correspond à une absence de pluie et le blanc à une pluie d'intensité maximale

On remarque alors comme illustré figure 21 que l'intensité des précipitations varie effectivement lorsque l'on se déplace d'une bande à une autre et qu'elle est d'autant plus importante que le niveau de gris est faible.

7

CONCLUSION

7.1 RÉSULTAT FINAL

L'objet de ce PSC était la réalisation d'un moteur de météo **locale** et **dynamique** en **temps réel** dans un **démonstrateur** ainsi que l'implémentation des **effets des conditions météorologiques** sur l'**environnement et les joueurs**.

En ce qui concerne le moteur de météo locale, nous avons d'abord testé puis écarté un modèle de simulation régi par les lois de la thermodynamique et de la **mécanique des fluides**. D'après nos essais sur Python, ce modèle est trop coûteux en temps de calcul pour une carte de grande dimension et des mailles élémentaires de petite échelle. Par ailleurs, les modèles que nous avons testés se généralisent mal à des nuages non-ponctuels. Ainsi ces modèles semblent peu adaptés à la reproduction des formes complexes et chaotiques des nuages. Pour traiter ce cas des nuages étendus, nous avons alors étudié une **approche probabiliste** reposant sur le principe d'automates cellulaires. Cependant, cette approche s'est révélée insatisfaisante de par la difficulté à contrôler les nuages et de par le temps de calcul trop élevé.

Ces raisons, ainsi qu'une réunion avec les développeurs de Microsoft Flight Simulator nous ont conduit à nous intéresser à d'autres méthodes permettant de résoudre à la fois les problèmes mis en évidence par la méthode physique et par l'approche probabiliste.

La solution de génération de météo locale et dynamique que nous proposons est une approche qui repose sur des méthodes de **génération procédurale**. Notre solution utilise un **bruit de valeur** (*value noise*) transformé et animé par des opérations mathématiques pour reproduire l'apparence des nuages. La gestion de l'éclairage et des ombres est rendue possible grâce à l'outil de rendu Volumetric Clouds. Afin d'augmenter l'impact sur le joueur, nous avons également ajouté une **carte de pluie** (*rain map*) qui spécifie les endroits où il pleut autour du joueur. Nous avons produit un **démonstrateur** sur Unity dans lequel il est possible d'évoluer à la première personne dans un monde virtuel où la météo est gérée selon la méthode retenue. Ce démonstrateur rend compte du caractère local et dynamique de la météo générée et permet de visualiser l'influence de la météo sur l'expérience utilisateur. Grâce à des variables introduites en entrée du shader, nous avons un certain contrôle sur la météo générée et notamment sur la couverture nuageuse, la pluie ou l'opacité des nuages.

7.2 RÉALISATION DES OBJECTIFS

Les objectifs de départ de notre projet étaient d’obtenir une météo locale, dynamique en temps réel et qui ait un impact sur le joueur. De plus, nous devons avoir un contrôle sommaire sur la météo affichée. Ces objectifs sont atteints:

- Notre modèle est affiché en **temps réel** grâce aux shaders permettant l’utilisation du GPU de l’ordinateur et la réduction du temps de calcul
- Pour le caractère **local**, notre simulateur contient des zones ayant des météos différentes (cumulonimbus, alto stratus, ciel dégagé) au sein d’une même carte
- Notre modèle est **dynamique** grâce à la superposition de bruits de valeur d’orientations différentes qui modifie légèrement la forme de nuages au cours du temps, tout en gardant un réalisme correct
- L’**impact sur le joueur** est obtenu grâce au pipeline de rendu HDRP permettant l’affichage des ombres des nuages, et par la pluie générée autour de lui. Cela limite la visibilité autour de lui et peut avoir une influence sur les décisions qu’il est amené à prendre.

7.3 PISTES D’AMÉLIORATION

A partir de ce modèle, nous pourrions implémenter d’autres effets de la météo ayant un impact sur le joueur tels que la neige ou le brouillard. Il est également possible d’augmenter encore le réalisme des nuages en jouant sur les octaves du bruit généré.

Une question abordée en fin de projet est celle de la gestion de la pluie et de ses effets. Nous générons en effet la pluie uniquement autour du joueur, pour des raisons d’optimisation et de limitation du nombre de particules présentes dans la scène. Si le joueur peut voir au loin un nuage dense et générateur de pluie, il ne verra cependant pas de particules de pluie sous celui-ci s’il ne se trouve pas en dessous du nuage. Nous pourrions aussi matérialiser l’effet de la pluie au sol avec un **changement de texture du sol**, un **ralentissement du joueur** et un **séchage progressif**.

Enfin, la suite naturelle de notre projet serait d’aborder les **transitions entre différents états de la météo**. Une des pistes que nous considérons est l’**interpolation** entre plusieurs cartes de couverture nuageuse pour gérer le passage d’une météo à une autre.

REFERENCES

- [1] R. Miyazaki S. Yoshida Y. Dobashi T. Nishita. “A Method for Modeling Clouds based on Atmospheric Fluid Dynamics”. In: Proceedings Ninth Pacific Conference on Computer Graphics and Applications. Pacific Graphics (2001). URL: <https://ieeexplore.ieee.org/document/962893>.
- [2] Y. Dobashi K. Kaneda H. Yamashita T. Okita T. Nishita. “A Simple, Efficient Method for Realistic Animation of Clouds”. In: (2000). DOI: 10.1145/344779.344795. URL: https://www.researchgate.net/publication/220721744_A_simple_efficient_method_for_realistic_animation_of_clouds.
- [3] Antoine Webanck. “Génération procédurale d’effets atmosphériques”. Ecole doctorale en Informatique et Mathématique de Lyon, 2019. URL: <https://www.theses.fr/2019LYSE1109>.
- [4] Alisson R. Silva, Adma R. Silva, and Maury M. Gouvêa. “A novel model to simulate cloud dynamics with cellular automaton”. In: *Environmental Modelling Software* 122 (2019), p. 104537. ISSN: 1364-8152. DOI: <https://doi.org/10.1016/j.envsoft.2019.104537>. URL: <https://www.sciencedirect.com/science/article/pii/S1364815218305000>.
- [5] *Procedural Weather Patterns*. 2018. URL: <https://nickmcd.me/2018/07/10/procedural-weather-patterns/>.
- [6] *Materials, Shaders and Textures in Unity*. URL: <https://docs.unity3d.com/560/Documentation/Manual/Shaders.html>.
- [7] *Shader Graph in Unity*. URL: <https://docs.unity3d.com/Manual/com.unity.shadergraph.html>.
- [8] *Value Noise*. 2018. URL: <https://www.ronja-tutorials.com/post/025-value-noise/>.
- [9] *Ray Tracing - NVIDIA Developer*. URL: <https://developer.nvidia.com/discover/ray-tracing>.
- [10] *Volumetric Clouds in Unity*. URL: <https://docs.unity3d.com/Packages/com.unity.render-pipelines.high-definition@12.0/manual/Override-Volumetric-Clouds.html>.
- [11] Jeff Johnson. “Rain Maker - 2D and 3D Rain Particle System for Unity”. In: 2022. URL: <https://assetstore.unity.com/packages/vfx/particles/environment/rain-maker-2d-and-3d-rain-particle-system-for-unity-34938>.